

0.92. Le snake, avec un menu — la couleur du serpent

Le jeu du 0.91.9, et un MENU entre le titre et la partie : GAUCHE et DROITE choisissent la couleur du serpent, A lance la partie.

1. Ce qu'il faut comprendre

C'est le jeu du 0.91.9 (le snake sans limite de temps), avec une chose en plus : un menu. Après le titre, START n'ouvre plus directement la partie : il ouvre le **MENU**. On y choisit la **couleur du serpent** avec GAUCHE et DROITE, puis **A** lance la partie. À la fin, START ramène au menu : on peut changer de couleur avant de rejouer.

Un quatrième écran : `ecran` valait 0 (titre), 1 (jeu) ou 2 (fin) ; il vaut maintenant aussi **3, le MENU**. La fonction `ouvrirMenu()` le dessine, comme `nouvellePartie()` dessine le jeu et `finPartie()` la fin.

Ce qui est nouveau ici : colorer le serpent, et lui seul. `couleurTexte()` colore **toutes** les lettres : les X et le P changeraient aussi. On se sert donc des **palettes** du 0.89 : le serpent va dans la **palette 1**, tout le reste reste dans la palette 0. Après chaque `poser()` du A ou d'un O, un `teindre(colonne, ligne, 1)` (le 0.89.1) met cette case dans la palette 1.

Changer de couleur, c'est changer la palette, pas les cases. `couleurFond(1, 3, rouge, vert, bleu)` change la **teinte 3** de la palette 1, celle des lettres (le 0.89). Toutes les cases de la palette 1 prennent la nouvelle couleur **d'un coup**, sans rien redessiner. C'est ce que fait `choisirCouleur()` : un `if` par couleur, et le nom de la couleur écrit dans le menu.

La couleur est un numéro : `uint8_t couleur = 0;` 0 = vert, 1 = rouge, 2 = bleu, 3 = violet. Comme `sens` au 0.91.7 : on retient un numéro, et des `if` disent ce qu'il veut dire.

Les noms ont des espaces derrière : `"VERT "`, `"ROUGE "`, `"VIOLET"` ont tous 6 cases. Si l'on passait de VIOLET à VERT sans espaces, il resterait « VERTET » : les 2 dernières lettres de VIOLET : on efface avec des espaces, comme au chapitre 1.

DROITE : la couleur suivante. `couleur = (couleur + 1) % 4;` : 0 → 1 → 2 → 3, puis 3 + 1 = 4, et 4 % 4 = 0 : on revient au vert (le % du 0.9).

GAUCHE : la couleur d'avant. On voudrait `couleur - 1`, mais un `uint8_t` ne descend pas sous 0 : 0 - 1 donnerait 255 (le 0.91.6). L'astuce : `(couleur + 3) % 4`. Ajouter 3 puis garder le reste par 4, c'est reculer d'un. 2 : (2 + 3) % 4 = 5 % 4 = 1. 0 : (0 + 3) % 4 = 3, le violet.

chaque(200) dans le menu : un appui sur la croix dure plusieurs images. Si l'on regardait la croix à chaque image, un seul appui ferait tourner toutes les couleurs. On ne la regarde donc que 5 fois par seconde. Ce `chaque(200)` a son chronomètre à lui : il ne gêne pas le `chaque(250)` du serpent.

Le petit serpent du menu : trois O et un A, en (8, 9) à (11, 9), dans la palette 1. En changeant de couleur, on le voit changer tout de suite.

Deux précautions, parce qu'une case garde sa palette : `effacer()` enlève la **lettre**, pas la **palette** de la case. 1. `viderEcran()` remet toutes les cases dans la palette 0 (une boucle de plus, sur les 20 colonnes) : sinon, FIN

ou MENU seraient à moitié de la couleur du serpent. 2. Quand le P réapparaît au hasard, `teindre(px, py, 0)` : la case a pu être celle du serpent.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **3053 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	103 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
viderEcran()	74 o	← nouvellePartie(), finPartie(), ouvrirMenu() — ta fonction
nouvellePartie()	188 o	← main() — ta fonction
finPartie()	51 o	← main() — ta fonction
choisirCouleur()	120 o	← ouvrirMenu(), main() — ta fonction
ouvrirMenu()	145 o	← main() — ta fonction
main()	802 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer(), teindre() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	848 o	← CopierTuiles — la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte() — un texte gravé
"FIN"	3 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"START : LE MENU"	15 o	← texte() — un texte gravé
"VERT "	6 o	← texte() — un texte gravé
"ROUGE "	6 o	← texte() — un texte gravé

"BLEU "	6 o	← texte()	– un texte gravé
"VIOLET"	6 o	← texte()	– un texte gravé
"MENU"	4 o	← texte()	– un texte gravé
"COULEUR :"	9 o	← texte()	– un texte gravé
"GAUCHE - DROITE"	15 o	← texte()	– un texte gravé
"LA COULEUR CHAN..."	17 o	← texte()	– un texte gravé
"A : JOUER"	9 o	← texte()	– un texte gravé
"APPUIE SUR START"	16 o	← texte()	– un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser()	– une table gravée
GRAND_TABLE_0_1_1	12 o	← poser()	– une table gravée
GRAND_TABLE_0_2_2	12 o	← poser()	– une table gravée
GRAND_TABLE_0_3_3	12 o	← poser()	– une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 16	284 o sur place, dans texte_grand_0(), nouvellePartie(), ouvrirMenu(), main() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
teindre()	× 8	199 o sur place, dans viderEcran(), nouvellePartie(), ouvrirMenu(), main() + AdresseCase (10 o)
texte()	× 14	154 o sur place, dans nouvellePartie(), finPartie(), choisirCouleur(), ouvrirMenu(), main() + EcrireTexte (10 o)
effacer()	× 3	64 o sur place, dans viderEcran(), main() + AdresseCase (10 o), EffacerCases (12 o)
couleurFond()	× 5	60 o sur place, dans choisirCouleur(), main()
bouton()	× 9	45 o sur place, dans main() + LireManette (37 o)
nombre()	× 2	20 o sur place, dans finPartie(), main() + EcrireNombre (33 o)
hasard()	× 2	6 o sur place, dans main() + Hasard (25 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
viderEcran()	× 3	9 o sur place, dans nouvellePartie(), finPartie(), ouvrirMenu()
choisirCouleur()	× 3	9 o sur place, dans ouvrirMenu(), main()
ouvrirMenu()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()
nouvellePartie()	× 1	3 o sur place, dans main()
finPartie()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste> // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <effacer> // efface des cases, ou tout le fond
#include <teindre> // met une case du fond dans une palette
#include <poser> // pose une tuile sur une case du fond
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <texte> // écrit un texte à l'écran
#include <nombre> // écrit un nombre en chiffres
#include <couleurFond> // choisit une couleur d'une palette du fond
#include <couleurTexte> // choisit la couleur des lettres
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
#include <bouton> // lit un bouton de la manette
#include <chaque> // répond 1 toutes les n millisecondes
#include <hasard> // tire un nombre au hasard
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0; // 0 = le TITRE, 1 = le JEU, 2 = la FIN, 3 = le MENU (NOUVEAU)
```

3. UNE VARIABLE — X

```
uint8_t x = 5; // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8;          // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15;        // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8;         // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0;      // les pièces ramassées
```

8. UNE VARIABLE — AX

```
uint8_t ax = 5;         // la place du A AVANT son pas... (le 0.91.3)
```

9. UNE VARIABLE — AY

```
uint8_t ay = 8;         // ...pour l'y remettre s'il entre dans un mur
```

10. UN TABLEAU — QX

```
uint8_t qx[50];         // la queue (le 0.91.6), un TABLEAU de 50 cases 0 : les colonnes...
```

11. UN TABLEAU — QY

```
uint8_t qy[50];         // ...et les lignes. qx[0] : la case collée au A.
```

12. UNE VARIABLE — LONGUEUR

```
uint8_t longueur = 0;    // combien de cases la queue utilise (0 : le A seul)
```

13. UNE VARIABLE — SENS

```
uint8_t sens = 0;        // où va le A (le 0.91.7). 0 = droite, 1 = bas, 2 = gauche, 3 = haut
```

14. UNE VARIABLE — COULEUR

```
uint8_t couleur = 0;     // NOUVEAU : la couleur du serpent. 0 = vert, 1 = rouge, 2 = bleu, 3 = violet
```

15. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {
    for (uint8_t l = 0; l < 18; l++) {
        effacer(0, l, 20);
        // NOUVEAU : chaque case de la ligne revient dans la palette 0.
        // effacer() enlève la lettre, pas la palette de la case : sans ça,
        // le FIN et le MENU seraient en partie de la couleur du serpent.
        for (uint8_t c = 0; c < 20; c++) {
            teindre(c, l, 0);
        }
    }
}

// ---- Une partie qui commence, TOUT au départ (le 0.91.2) ----
// Depuis le titre, ET depuis la fin (START : rejouer).
```

16. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    x = 5;                                // le A revient à sa place
    y = 8;
    px = 15;                              // le P revient en (15, 8), d'où qu'il soit
    py = 8;
    ax = 5;                                // et sa place d'avant, la même
    ay = 8;
    longueur = 0;                          // pas de queue, le A est tout seul (le 0.91.6)
    sens = 0;                              // le A repart vers la droite (le 0.91.7)
    score = 0;                             // pas encore de pièce
    ecran = 1;                             // on est sur le JEU
    viderEcran();
    poser(px, py, ALPHABET[15]);
    poser(x, y, ALPHABET[0]);              // le A (le 0.91.7)...
    teindre(x, y, 1);                      // NOUVEAU : ...dans la palette 1, celle du serpent

    // ---- Le cadre de murs, des X tout autour du terrain (le 0.91.3) ----
    //
    // colonne : 0 1 2 ...                19
    // ligne 0 : X X X X X X ... X X X    le haut : ligne 0, colonnes 0 à 19
    // ligne 1 : X                        X
    // ...      X  A      P  X          les côtés : colonnes 0 et 19
    // ligne 15: X                        X
    // ligne 16: X X X X X X ... X X X    le bas : ligne 16
    // ligne 17: SCORE 000                la ligne 17 reste au score
    //
    // ALPHABET[23] : la 24e lettre (on compte depuis 0), le X.
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 0, ALPHABET[23]);        // le haut
        poser(c, 16, ALPHABET[23]);       // le bas
    }
    for (uint8_t l = 1; l < 16; l++) {
        poser(0, l, ALPHABET[23]);         // le côté gauche
        poser(19, l, ALPHABET[23]);        // le côté droit
    }

    texte(0, 17, "SCORE");                 // plus de TEMPS à côté (le 0.91.9)
}

// ---- Une partie qui s'arrête, l'écran FIN (le 0.91.8) ----
// Une seule raison de perdre : le mur (le 0.91.9).
```

17. UNE FONCTION — FINPARTIE()

```
void finPartie() {
    ecran = 2;                             // on est sur la FIN
    viderEcran();
    texte(8, 6, "FIN");
    texte(5, 9, "SCORE");                  // le score de la partie (le 0.91.2)
    nombre(11, 9, score);
    texte(2, 13, "START : LE MENU");       // NOUVEAU : on repasse par le menu
}

// ---- NOUVEAU : la couleur du serpent ----
// Le serpent (le A et ses 0) est dans la PALETTE 1 (teindre, le 0.89.1).
// Changer la couleur, c'est changer la teinte 3 de la palette 1 (celle des
// lettres) : TOUTES les cases de la palette 1 changent d'un coup.
// Et on écrit le nom de la couleur dans le menu, avec des espaces derrière
// pour recouvrir un nom plus long (VIOLET a 6 lettres, VERT n'en a que 4).
```

18. UNE FONCTION — CHOISIRCOULEUR()

```

void choisirCouleur() {
    if (couleur == 0) {
        couleurFond(1, 3, 0, 24, 0);           // rouge 0, vert 24, bleu 0 : VERT
        texte(11, 6, "VERT ");
    }
    if (couleur == 1) {
        couleurFond(1, 3, 31, 0, 0);           // tout rouge : ROUGE
        texte(11, 6, "ROUGE ");
    }
    if (couleur == 2) {
        couleurFond(1, 3, 0, 8, 31);           // surtout du bleu : BLEU
        texte(11, 6, "BLEU ");
    }
    if (couleur == 3) {
        couleurFond(1, 3, 20, 0, 31);          // bleu + un peu de rouge : VIOLET
        texte(11, 6, "VIOLET");
    }
}

// ---- NOUVEAU : l'écran du MENU ----
//
// ligne 2 :          MENU
// ligne 6 :  COULEUR : VERT
// ligne 9 :          OOOA          un petit serpent, pour voir la couleur
// ligne 12:  GAUCHE - DROITE
// ligne 13:  LA COULEUR CHANGE
// ligne 16:   A : JOUER

```

19. UNE FONCTION — OUVRIRMENU()

```
void ouvrirMenu() {
    ecran = 3; // on est sur le MENU
    viderEcran();
    texte(8, 2, "MENU");
    texte(1, 6, "COULEUR :");

    // le petit serpent d'exemple : trois 0 et un A, tous dans la palette 1
    for (uint8_t c = 8; c < 11; c++) {
        poser(c, 9, ALPHABET[14]);
        teindre(c, 9, 1);
    }
    poser(11, 9, ALPHABET[0]);
    teindre(11, 9, 1);

    texte(2, 12, "GAUCHE - DROITE");
    texte(1, 13, "LA COULEUR CHANGE");
    texte(5, 16, "A : JOUER");
    choisirCouleur(); // la couleur choisie, et son nom
}
```

20. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            ouvrirMenu();                // NOUVEAU : le titre mène au MENU
        }

        // ---- NOUVEAU : dans le MENU, la croix choisit la couleur ----
        // chaque(200) : on ne regarde la croix que 5 fois par seconde. Sans lui,
        // un appui (qui dure plusieurs images) ferait défiler toutes les couleurs.
        if (ecran == 3 && chaque(200)) {
            if (bouton(DROITE)) {
                couleur = (couleur + 1) % 4; // la suivante : 0 1 2 3, puis 0
                choisirCouleur();
            }
            if (bouton(GAUCHE)) {
                couleur = (couleur + 3) % 4; // la précédente : 3 2 1 0, puis 3
                choisirCouleur();           // (+ 3 puis % 4 : comme - 1, sans passer sous 0)
            }
        }
        if (ecran == 3 && bouton(A)) {
            nouvellePartie();            // A : on joue, avec cette couleur
        }

        if (ecran == 1) {
            // ---- La croix CHOISIT la direction (le 0.91.7) ----
            // On appuie une fois, même très court : sens change, et il le reste.
            if (bouton(DROITE)) sens = 0;
            if (bouton(BAS))    sens = 1;
            if (bouton(GAUCHE)) sens = 2;
            if (bouton(HAUT))   sens = 3;

            ax = x;                // on retient où est le A... (le 0.91.3)
            ay = y;

            // ---- Le A avance TOUT SEUL, un pas toutes les 250 ms (le 0.91.7) ----
            // chaque(250) : vrai 4 fois par seconde (le 0.77).
            //
            //      sens :    3
            //                ↑      colonne : x - 1 à gauche, x + 1 à droite
            //      2 ← A → 0      ligne   : y - 1 en haut, y + 1 en bas
            //                ↓
            //                1
            if (chaque(250)) {
                effacer(x, y, 1);        // le A quitte sa case
                if (sens == 0) x = x + 1; // à droite
                if (sens == 1) y = y + 1; // en bas
                if (sens == 2) x = x - 1; // à gauche
                if (sens == 3) y = y - 1; // en haut
                poser(x, y, ALPHABET[0]); // et se pose sur la suivante,
                teindre(x, y, 1);         // NOUVEAU : dans la palette 1
            }

            // ---- Le A touche le cadre ? La partie s'arrête (le 0.91.8) ----
            // Avant (le 0.91.3), le A reculait. Maintenant, il a PERDU.
            // || veut dire « OU » : une seule des quatre suffit.
            if (x == 0 || x == 19 || y == 0 || y >= 16) {
                finPartie();            // l'écran FIN, avec le score
            }
        }
    }
}
```

```

    continue; // on saute TOUT le reste de ce tour
} // de boucle : retour à image()

// ---- La queue suit le A (le 0.91.5) ----
// Le A a-t-il bougé ? Sa place n'est plus celle d'avant (ax, ay).
// longueur > 0 d'abord (le 0.91.6). Sans queue, rien à déplacer, et
// longueur - 1 vaudrait 255 (un uint8_t ne descend pas sous 0).
// && veut dire « ET » : il faut une queue ET que le A ait bougé.
if (longueur > 0 && (x != ax || y != ay)) {
    // 1. le BOUT de la queue disparaît : la dernière case, longueur - 1
    effacer(qx[longueur - 1], qy[longueur - 1], 1);

    // ---- 2. chaque case prend la place de celle de devant ----
    // En partant du BOUT, et en reculant (i-- : i = i - 1).
    // Avec longueur = 3 : i = 2 : qx[2] = qx[1]
    //                      i = 1 : qx[1] = qx[0]
    //                      i = 0 : 0 > 0 est faux, la boucle s'arrête.
    for (uint8_t i = longueur - 1; i > 0; i--) {
        qx[i] = qx[i - 1];
        qy[i] = qy[i - 1];
    }

    // 3. la première case prend la case que le A vient de quitter
    qx[0] = ax;
    qy[0] = ay;

    // 4. toute la queue redessinée, de la case 0 à la case longueur - 1
    // (après une pièce, la nouvelle case venait d'être effacée avec le bout)
    for (uint8_t i = 0; i < longueur; i++) {
        poser(qx[i], qy[i], ALPHABET[14]);
        teindre(qx[i], qy[i], 1); // NOUVEAU : chaque 0 dans la palette 1
    }
    poser(x, y, ALPHABET[0]); // 5. le A par-dessus (le 0.91.5)
    teindre(x, y, 1);
}
if (x == px && y == py) { // le A sur le P : ramassée (le 0.81)
    score = score + 1;

    // ---- La pièce réapparaît AU HASARD, mais DANS le cadre (le 0.91.4) ----
    //
    // px = 1 + hasard() % 18;
    // | | |
    // | | +-- % 18 : le reste de la division par 18, de 0 à 17.
    // | | Exemple : hasard() rend 137 ; 137 = 7 × 18 + 11 ;
    // | | le reste est 11.
    // | +----- un nombre imprévisible, de 0 à 255 (le 0.81.2)
    // +----- + 1 : on décale de 0...17 à 1...18 → 11 + 1 = 12.
    //
    // Pourquoi pas % 20 comme au 0.81.2 ? Les colonnes 0 et 19 sont des murs X :
    // la pièce doit rester de 1 à 18. Les lignes 0 et 16 aussi sont des murs :
    // la pièce doit rester de 1 à 15, d'où 1 + hasard() % 15.
    px = 1 + hasard() % 18; // une colonne : 1 à 18
    py = 1 + hasard() % 15; // une ligne : 1 à 15
    poser(px, py, ALPHABET[15]); // le nouveau P apparaît là,
    teindre(px, py, 0); // NOUVEAU : palette 0. La case a pu être
    // au serpent : le P serait de sa couleur.

    // ---- La queue grandit d'une case (le 0.91.6) ----
    // La nouvelle case naît SUR le bout de la queue. Au prochain pas, toutes
    // les autres avancent, elle non : elle reste derrière.
    // La toute première n'a pas de bout : elle naît SOUS le A.
    // < 50 : le tableau n'a que 50 cases, qx[0] à qx[49].
    if (longueur < 50) {
        if (longueur == 0) {
            qx[0] = x; // la première case : sous le A
            qy[0] = y;

```

```

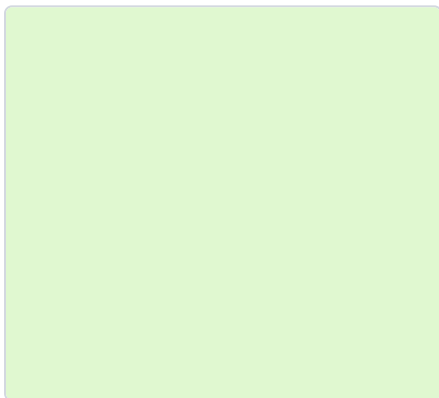
    } else {
        qx[longueur] = qx[longueur - 1]; // la case d'après le bout...
        qy[longueur] = qy[longueur - 1]; // ...à la même place que le bout
    }
    longueur = longueur + 1; // une case de plus
}
}
nombre(6, 17, score);
}

// ---- Sur la fin, START (le 0.91.2) : NOUVEAU, retour au MENU.
if (ecran == 2 && bouton(START)) {
    ouvrirMenu(); // on peut changer de couleur avant de rejouer
}
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

JEU

APPUIE SUR START

A — l'écran n'a pas bougé

JEU

APPUIE SUR START

DROITE — l'écran n'a pas bougé

JEU

APPUIE SUR START

GAUCHE — l'écran n'a pas bougé

JEU

APPUIE SUR START

HAUT — l'écran n'a pas bougé

JEU

APPUIE SUR START

BAS — l'écran n'a pas bougé

MENU

COULEUR : VERT

000A

GAUCHE - DROITE
LA COULEUR CHANGE

A : JOUER

START — l'écran a changé

5. Ce qu'on doit voir

START : le MENU ; GAUCHE et DROITE changent la couleur du petit serpent ; A : on joue, le serpent a cette couleur, le reste non.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ START : le MENU, VERT, le petit serpent dans la palette 1
- ✓ DROITE : ROUGE, et la palette 1 devient rouge
- ✓ GAUCHE deux fois : de 1 à 0, puis de 0 à 3, VIOLET
- ✓ A : le jeu ; le A et sa queue dans la palette 1, le P et les X dans la 0 (x = 16)
- ✓ au mur : FIN, et plus aucune case dans la palette 1
- ✓ le fond de la palette 1 est celui de la palette 0 (28,31,26 / 28,31,26)

Ensuite : **0.92.1. Le snake, avec un menu — le pas compté en images** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++